

Projet – pédagogie active

Inventer une variante d'un problème classique de concurrence

Du lore d'origine à une implémentation C++20 – en binôme

En une phrase

Vous recevez un problème classique de synchronisation, vous en étudiez la structure, vous la transposez dans un nouveau lore de votre invention, puis vous l'implémentez en C++20 en deux versions : l'une correcte, l'autre volontairement interbloquée.

Travail en
Volume
Évaluation
Dépôt

binômes (groupes de 2)
2 séances de 4h – soit 8 h encadrées
20 points répartis sur 5 livrables, bonus possibles
Cours AMeTICE "[26-27]-R5.05 Prog. concurrente"

Les problèmes étudiés sont issus de *The Little Book of Semaphores* d'Allen B. Downey (chapitre 5), disponible sur le cours AMeTICE ou au lien suivant : <https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf>.



1. Ce qu'on vous demande, et pourquoi

Les problèmes classiques de la programmation concurrente – les philosophes, les lecteurs et les rédacteurs, le producteur et le consommateur – ont deux caractéristiques utiles. Leur *lore*, c'est-à-dire l'histoire dans laquelle ils sont racontés, est arbitraire : les philosophes pourraient être des imprimantes, les fourchettes des jetons de licence. Mais la **structure** qui se cache derrière, elle, ne l'est pas : c'est elle qui rend le problème intéressant, et c'est elle qu'on retrouve dans les vrais systèmes.

Séparer les deux est précisément la compétence que ce projet évalue. Vous saurez que vous avez compris un problème de concurrence le jour où vous serez capable de le raconter autrement sans rien lui enlever.

Objectifs

- Identifier et expliquer les concepts de concurrence mobilisés par un problème donné.
- Distinguer ce qui est accessoire (le décor) de ce qui est essentiel (les contraintes de synchronisation).
- Concevoir une variante originale qui préserve cette structure.
- Implémenter, tester et documenter une solution en C++20, ainsi qu'une version volontairement fautive.
- Restituer le travail à l'oral, en dix minutes, devant la promotion.

1.1. Organisation

Le travail se fait en **binômes**. Chaque binôme travaille sur **un problème différent**, attribué avant la première séance. L'attribution résulte d'un **vote par classement** : chaque binôme ordonne les problèmes d'une présélection par ordre de préférence, et les attributions sont calculées à partir de ces classements.

Votre problème

La présélection est faite chaque année dans le **chapitre 5 du *Little Book of Semaphores*** d'Allen B. Downey (*Less classical synchronization problems*). Le chapitre vous est fourni. Il contient, pour chaque problème, l'énoncé, un indice et la solution de l'auteur en pseudo-code.

Liste retenue cette année : 8 problèmes de la section 5 + problème 4.5.

Modalités du vote : 3 vœux à envoyer par mail avant la fin de la semaine à alexis.guyot@univ-amu.fr.

Une précision importante sur le livre

Downey donne la solution de chaque problème. Ce n'est pas un problème pour ce projet : votre travail ne consiste pas à *trouver* la solution, mais à *comprendre* pourquoi elle fonctionne, à l'abstraire, puis à la transposer. Une transposition mécanique du pseudo-code sans compréhension se voit immédiatement, que ce soit au livrable 2, à l'oral, ou aux questions.

2. Usage de l'intelligence artificielle générative

Usage interdit

L'usage de l'IA générative est **interdite** pour le projet. Transgresser cette règle mènera inéluctablement à une saisine du conseil de discipline, sans négociation possible.

3. D  roul  

Le projet se d  roule en cinq phases. Les quatre premi  res produisent chacune un livrable   crit ou logiciel, la cinqui  me une restitution orale.

Phase	Objet	Livrable	��ch��ance
1	Comprendre les concepts	L1 – glossaire	Voir AMeTICE
2	Extraire la structure	L2 – caract��ristiques fondamentales	Voir AMeTICE
3	Changer de lore	L3 – nouveau probl��me, deadlock, tableau de correspondance	Voir AMeTICE
4	Impl��menter	L4 – projet Qt Creator, deux versions, tests	Voir AMeTICE
5	Restituer	L5 – pr��sentation orale et diapositives	Voir AMeTICE

D  coupage indicatif du temps encadr   : 3 h pour les phases 1    3, 3 h pour la phase 4, 2 h pour la finalisation et la pr  paration de l’oral.

3.1. Phase 1 – Identifier et comprendre les concepts

Votre probl  me mobilise un certain nombre de concepts de programmation concurrente. Certains apparaissent explicitement dans l’  nonc  , d’autres sont cach  s dans la solution.

- Listez l’**ensemble** des concepts impliqu  s par le probl  me et par sa solution.
- Ajoutez-y la notion d’**interblocage** (*deadlock*), qui fait partie du glossaire quel que soit votre probl  me.
- Pour chacun, produisez une d  finition qui soit    la fois **juste** et **compr  hensible** par un   tudiant de BUT1 ou 2 qui n’a pas suivi ce module.

Livrable 1 – Glossaire des concepts

Un document listant chaque concept mobilis  , avec sa d  finition et, quand c’est utile, un exemple tir   de votre probl  me.

Format : Libre, dans un fichier PDF (.pdf). Peut   tre une liste type <Concept> : <D  finition>. [Ex: <Exemple>], ou un tableau    2 ou 3 colonnes.

Sur la vulgarisation

Une d  finition qui recopie b  tement le cours, Wikip  dia ou la premi  re r  ponse d’une IA n’est pas une d  finition comprise. La vulgarisation est un exercice difficile : une d  finition trop courte peut   tre incompl  te et incompr  hensible sans bagage suppl  mentaire ; et,    l’inverse, une d  finition trop longue peut perdre votre interlocuteur avec des d  tails superflus. Prenez le temps de faire l’exercice, et prenez du recul par rapport    vos productions.

3.2. Phase 2 – Extraire la structure du probl  me

C’est la phase la plus exigeante du projet, et celle qui d  termine la qualit   de tout ce qui suit.

1. **Listez les caract  ristiques** de votre probl  me : les acteurs, les ressources, les contraintes de synchronisation, les conditions d’attente, les propri  t  s attendues.
2. **Triez-les** : lesquelles sont essentielles – c’est-  -dire que les supprimer changerait la nature du probl  me – et lesquelles rel  vent du d  cor ?
3. **G  n  ralisez** les caract  ristiques essentielles : formulez-les sans employer aucun mot du lore d’origine.

Le test de la généralisation

Une caractéristique correctement généralisée ne contient plus ni barbier, ni marmite, ni renne.
« Le service ne peut commencer que si un client et un serveur sont simultanément disponibles » est généralisé; « le barbier ne coupe que s'il y a un client » ne l'est pas.

Livrable 2 – Caractéristiques fondamentales généralisées

La liste des caractéristiques essentielles de votre problème, formulées indépendamment du cadre applicatif d'origine.

Format : Liste au format libre dans un fichier PDF (.pdf).

3.3. Phase 3 – Changer de lore

Vous inventez maintenant un nouveau cadre applicatif pour la structure que vous venez d'isoler.

- **Définissez le nouveau problème** : qui sont les acteurs, quelles sont les ressources, que cherche-t-on à faire ?
- **Identifiez une situation d'interblocage** possible dans ce nouveau cadre, et **démontrez** qu'il s'agit bien d'un interblocage.
- **Montrez que les caractéristiques fondamentales sont préservées**, une par une.

Blocage n'est pas interblocage

Un programme qui ne progresse plus n'est pas nécessairement en situation d'interblocage. Un interblocage est un état où un ensemble de tâches s'attendent mutuellement, sans qu'aucune ne puisse jamais progresser. Pour le démontrer, exhibez le scénario complet et montrez que les **quatre conditions de Coffman** sont réunies.

Livrable 3 – Le nouveau problème

Un document en trois sections :

1. description du nouveau problème ;
2. description de l'interblocage, avec la preuve qu'il s'agit bien d'un interblocage ;
3. tableau à trois colonnes : chaque caractéristique fondamentale du livrable 2, sa déclinaison dans le lore d'origine, sa déclinaison dans le nouveau lore.

Format : Libre, dans un fichier PDF (.pdf).

3.4. Phase 4 – Implémenter

- Implémentez une **solution correcte** de votre nouveau problème, sans interblocage.
- **Clonez-la** et introduisez-y l'interblocage décrit au livrable 3.
- Écrivez des **tests** qui prouvent le comportement de chacune des deux versions : au moins un pour la version correcte, au moins un pour la version fautive.
- **Commentez** le code et **expliquez les tests** : leur(s) objectif(s) et les résultats / observations attendu(e)s.

Langage et outils : **C++20**, sous **Qt Creator**. Vous disposez de tout ce qui a été vu en cours et en TP : `std::jthread`, `std::mutex` et les verrous RAII, `std::atomic`, `std::counting_semaphore`, `std::condition_variable`. Le choix des bons outils fait partie du travail et sera discuté à l'oral.

Comment teste-t-on une version qui se bloque ?

C'est la bonne question, et elle se pose dans les deux sens. Pour la version correcte, il faut montrer que le programme termine, même dans des conditions difficiles/particulières, *et* que

l'invariant est respecté. Pour la version fautive, un test qui attend indéfiniment n'est pas un bon test : utilisez des `print` et la console pour montrer que vous aboutissez bien à un interblocage.

Une exécution ne prouve rien

Un bug de concurrence peut ne se manifester qu'une fois sur mille. Un test qui passe une fois ne démontre rien : répétez, faites varier le nombre de threads, et dites honnêtement dans vos commentaires ce que votre test établit et ce qu'il n'établit pas.

Livrable 4 – Projet Qt Creator

Un projet contenant les deux implémentations et les tests, **commentés**, ainsi que les résultats d'exécution attendus. L'organisation du code est libre : un fichier unique convient si le code reste court et lisible, sinon découpez.

Format : Projet QtCreator zippé (.zip), uniquement avec les sources (pas de binaires).

3.5. Phase 5 – Restituer

Présentation devant la promotion : **10 minutes au total**, dont **6 minutes d'exposé** et **4 minutes de questions**. La structure suivante est imposée.

1. Introduire le problème d'origine avec son lore.
2. Présenter les concepts de concurrence sous-jacents : un seul concept central est développé – **déterminé avec l'enseignant** – les autres sont simplement mentionnés.
3. Décrire le nouveau lore.
4. Présenter l'interblocage imaginé.
5. Mentionner les outils C++20 utilisés.
6. Conclure par une **unique** diapositive de synthèse.

Six minutes, c'est court

Six points à couvrir en six minutes : chacun tient en une diapositive et en plus ou moins une minute. **C'est un exercice de synthèse**. Les détails sont couverts par les autres livrables intermédiaires. Répétez avec un chronomètre. Si vous dépassez, **vous serez coupés** sans sommation.

Livrable 5 – Présentation

Les diapositives, déposées avant l'oral, l'exposé lui-même et les réponses aux questions.

Format et échéance de dépôt : Slides au format PDF (.pdf), à déposer au maximum la veille des oraux.

4. Évaluation

Le total est de 20 points. Certains critères permettent de dépasser l'enveloppe annoncée : ce sont des **bonus**.

Livrable	Critère	Points
L1 – Glossaire (/2,5)	Tous les concepts sont identifiés : non (0) / oui	1
	Qualité des définitions : non rendu ou trop mauvaise (0), qualité variable, manque de clarté, phrases vides de sens (0,5 à 1), claires et traduisant une bonne compréhension	1,5
L2 – Caractéristiques (/2)	Toutes les caractéristiques fondamentales importantes sont identifiées : non (0) / oui	1
	Qualité de la généralisation : pas fait ou trop mauvaise (0), passable (0,5), bonne	1
L3 – Nouveau lore (/2,5)	Qualité du nouveau lore : pas fait (0), pas totalement adapté ou description peu claire (0,5 à 1), préserve l'ensemble des caractéristiques fondamentales	1,5
	Qualité de l'interblocage : pas fait, hors sujet ou vide de sens (0), blocage mais pas interblocage (0,25), correct	1
	<i>Bonus</i> originalité et créativité	+0,5
L4 – Code (/7)	Code qui compile : version sans interblocage (1), version avec interblocage (1)	2
	Tests de la version sans interblocage : pas faits (0), insuffisants (0,5), suffisants	1
	Tests de la version avec interblocage : pas faits (0), insuffisants (0,5), suffisants	1
	Qualité du code – nommage, découpage en fonctions, organisation : non rendu ou catastrophique (0), passable (0,5 à 1,5), bonne	2
	Qualité des commentaires : aucun (0), passable – inutiles, vides de sens, trop peu nombreux (0,5), facile à lire et à prendre en main	1
L5 – Oral (/6)	Respect de la structure imposée : non ou pas fait (0), oui	1
	Esprit de synthèse : pas fait ou très mauvais (0), insuffisant dans l'ensemble (0,5), insuffisant sur certaines parties (1), bon dans l'ensemble	1,5
	Qualité des diapositives : non rendu ou catastrophique (0), forme soignée avec trop de fautes (0,5), forme soignée sans faute	1
	Réponses aux questions : pas fait ou très insuffisant (0), insuffisant ou trop peu clair (0,5), suffisant (1,5), très bien	2
	Respect du temps de parole : non ou pas fait (0), mauvaise répartition (0,5), oui	1

Remarque

Le code pèse 7 points sur les 20. Prenez le temps de réfléchir avant de vous lancer, et de prendre du recul sur la qualité de vos productions à chaque étape avant de passer à la suite. Vous êtes en troisième année maintenant : montrez que vous êtes plus que des cracheurs de code. Toutes les compétences travaillées dans ce projet vous seront utiles dans votre futur professionnel, voire même personnel.

5. Calendrier et modalités pratiques

Séance	Date	Objet
Avant S1	Semaine du 28/09	Vote par classement, attribution des problèmes
Séance 1	Semaine du 05/10	Phases 1 à 3 – analyse
Séance 2	Semaine du 12/10	Phase 4 – implémentation, finalisation, préparation de l'oral
Soutenances	Semaine du 19/10	Phase 5 – restitutions

Dépôt. Cours AMeTICE "[26-27]-R5.05 Prog. concurrente". *Les deadlines par livrable sont fixées là-bas.*

Retards. 0 pour le livrable.

Référence : Allen B. Downey, *The Little Book of Semaphores*, Green Tea Press, chapitre 5 *Less classical synchronization problems*. Ouvrage diffusé sous licence libre.